

UNIVERSITY OF BUEA

REPUBLIC OF CAMEROON

FACULTY OF ENGINEERING
AND TECHNOLOGY

PEACE-WORK-FATHERLAND

DEPARTMENT OF COMPUTER
ENGINEERING



COURSE TITLE:

**INTERNET PROGRAMMING (J2EE) AND MOBILE
PROGRAMMING**

System Modelling and Design

COURSE CODE: CEF 440

NAMES	MATRICULE
BERINYUY GILLES KILALON	FE23A024
ITIE LLOYD BONYEKI	FE23A069
NGUEFACK ELVIRA DIKIELLE	FE23A112
NKWAIN REMI KUMA	FE23A123
NKENGLEFAC DORENOEL	FE23A120

**COURSE INSTRUCTOR
DR. NKEMENI VALERY**

Table of Contents

INTRODUCTION:	3
What are Context Diagrams?	3
Importance of Context Diagrams in Systems Analysis	3
Identified Elements of the Context Diagram	3
Figure 1 Context Diagram	5
Data Flow Diagram	5
Level 0 – Context Diagram	5
Level 1 – Process Decomposition	6
Use Case Diagram	8
Actors	8
Use Cases and Relationships	8
Figure 3 Use Case Diagram	9
Sequence Diagram	10
Scenario S1: Adaptive Content Delivery	10
.....	10
Scenario S2: Quiz Taking When Internet Drops	10
.....	11
Scenario S3: Instructor Course Creation with Video Upload	11
Scenario S4: Manual Format Selection by Student	12
.....	13
Scenario S5: Concurrent Live Lecture	13
.....	14
Scenario S6: Complete Offline Mode	14
.....	15
Scenario S7: Quality of Experience Analytics	15
.....	16
Class Diagram	17
Classes	17
Figure 4 Class Diagram	Error! Bookmark not defined.
CONCLUSION:	20

INTRODUCTION:

To ensure a robust and scalable architecture, various system modelling techniques have been applied. These include context diagrams, data flow diagrams, use case diagrams, sequence diagrams, class diagrams, and deployment diagrams. Each diagram provides a different perspective on the system, helping to visualize how components interact, how data flows, how users engage with the system, and how the application is structured and deployed.

The modelling and design tasks documented in this report form a critical foundation for the implementation and testing of the mobile application. They enable stakeholders to understand system functionality, identify potential issues early, and guide the development process in a structured and efficient manner.

What are Context Diagrams?

Context Diagrams are high-level visual representations that show the interactions between a system being developed and its external entities, such as users, other systems, or processes. They provide a big-picture view of how the system fits into its environment without diving into the internal details of the system itself.

- Typically, they consist of a central system surrounded by external entities, with arrows representing data flow or interactions between them.
- They're useful for understanding system boundaries and dependencies.

Importance of Context Diagrams in Systems Analysis

- **Scope Definition:** Context diagrams define the system's boundaries by highlighting its interactions with external entities, ensuring that the analysis focuses on pertinent components and processes.
- **Requirement Gathering:** These diagrams visualize how the system interacts with its environment, aiding in identifying both functional and non-functional requirements. They offer clarity on the system's objectives and its external interactions.
- **Communication:** Acting as a bridge between stakeholders, such as business users, developers, and project managers, context diagrams foster shared understanding of the system's scope and context. They streamline discussions and decision-making throughout the development process.
- **Risk Identification:** Context diagrams assist in spotting potential risks stemming from the system's interactions with external entities. They help stakeholders assess the implications of external factors on the system's performance, security, and reliability.

Identified Elements of the Context Diagram

Student: An External entity. Unambiguously outside the system. They interact with it but are not part of it.

Instructor External entity. Also, outside the System and interact with the system to achieve their objectives.

University / Institution From analysis and critical thinking, the university do not actually interact with the system directly. In our context, the university is more of a **sponsor or adopter** decision-maker. They don't log in, they don't send data, they don't receive automated reports from the system. Their "interaction" is an offline decision to adopt the platform. That means they have **no data flow** with the system, and anything without a data flow has no business on a context diagram.

System Administrator : An admin who logs into the platform to manage users and configure settings is just another **user role**, meaning they interact with the system the same way a student or instructor does, just with elevated permissions. They are not a separate external system. In DFD terms, they'd be a separate actor because their data flows are different from a student's, but they're still a human user of the platform.

Streaming Server (HLS) Our stack already uses HLS-based adaptive streaming. The question which was bothering the team is: are we going to be hosting the streaming server Ourselves, or using a third-party CDN like Cloudflare Stream or AWS MediaLive? We could be running our own Node.js/FFmpeg HLS server as part of the backend, so it's **internal**. The alternative is using an external CDN or media service. Given that this is a student project where we will be controlling the full stack, the streaming logic almost certainly lives inside the backend.

Transcription API : We decided to integrate something like Google Speech-to-Text or AssemblyAI. That makes it genuinely external.

Notification Service We decided to use FCM as it is reasonable and lightweight integration same as the transcription API.

Summary of decisions

Entity	Decision	Reason
Student	Keep	Clear external actor with direct data flows
Instructor	Keep	Clear external actor with direct data flows
University / Institution	Remove	No direct data flow with the system
System Administrator	Keep	User role with distinct data flows
Streaming Server	Remove	Internal process in our backend
Transcription API	Keep	Realistic third-party integration
ISP / Network	Remove	Environmental factor, not a data-exchanging entity
Notification Service	Keep	Realistic third-party integration (FCM)

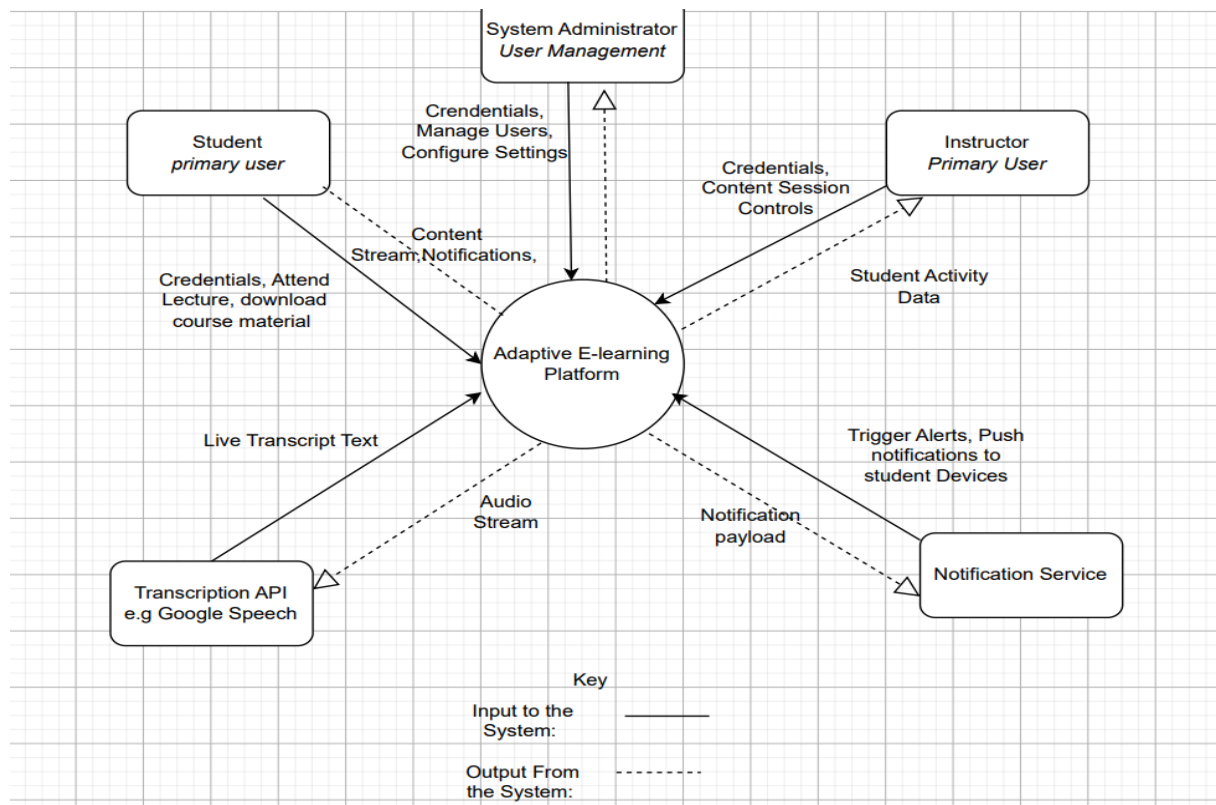


Figure 1 Context Diagram

Data Flow Diagram

A Data Flow Diagram (DFD) is a system modeling tool used in software engineering to represent the flow of data within a system. It was developed as part of structured analysis and uses a set of standard symbols to show how data is received, processed, stored, and sent out.

DFDs are drawn at different levels of detail. Level 0, also called the Context Diagram, shows the system as a whole and its interaction with the outside world. Level 1 breaks the system down into its main internal processes and data stores.

Level 0 – Context Diagram

The Level 0 diagram, also called the Context Diagram, shows the entire platform as one single process. It focuses on what comes in and what goes out. The four external entities that interact with the system are the Student, the Instructor, the Admin, and the Network Service.

1 Student

The Student is the primary user of the platform. The data the student sends into the system includes login credentials, content requests, and quiz submissions. The data the student receives from the system includes course content, quiz scores, progress reports, and notifications.

2 Instructor

The Instructor is responsible for creating and managing course materials. The data the instructor sends into the system includes course videos, notes, quizzes, and any edits or deletions to

existing content. The data the instructor receives from the system is a confirmation that the upload or update was successful

3 Admin

The Admin manages the users of the platform. The data the admin sends into the system includes user management requests such as creating or deleting accounts. The data the admin receives from the system includes QoE (Quality of Experience) reports and system logs that show how the platform is performing.

4 Network Service

The Network Service is an external service that monitors the internet connection. The data it sends into the system includes bandwidth levels, latency values, and packet loss rates. The system uses this data to decide how to deliver content to the student. The Network Service does not receive data back from the system in a meaningful way it is mainly a data provider.

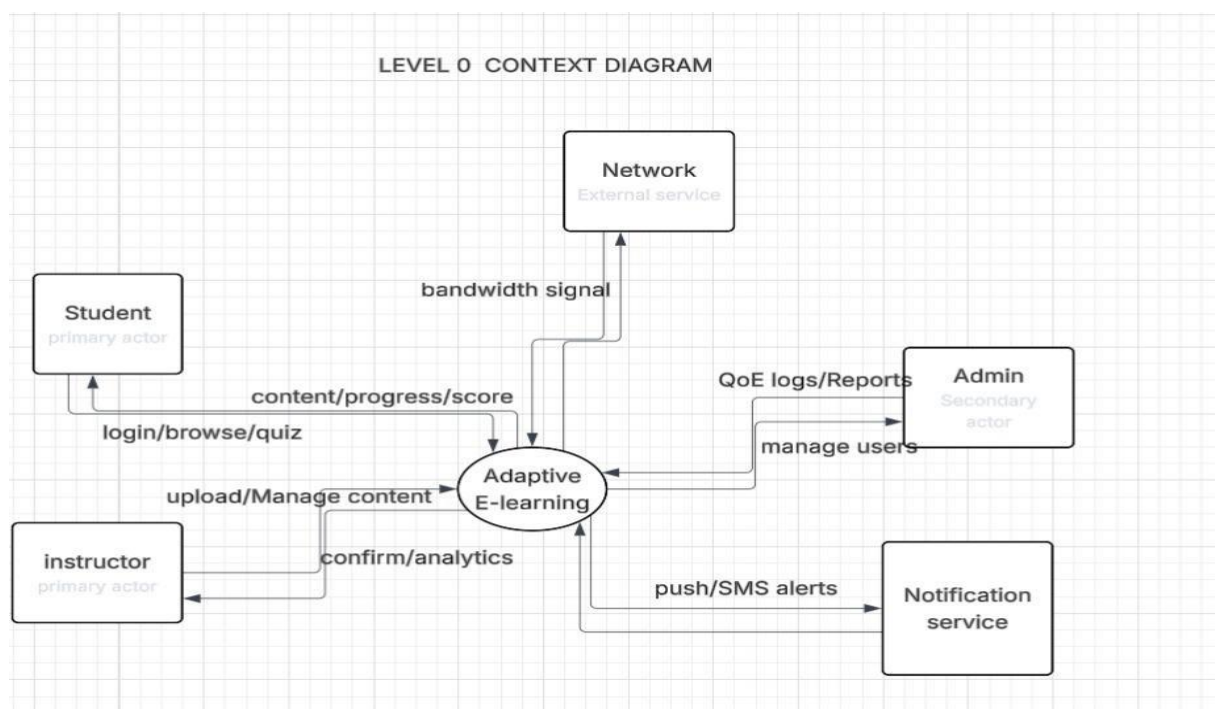


Figure 2 level 0 Data Flow

Level 1 – Process Decomposition

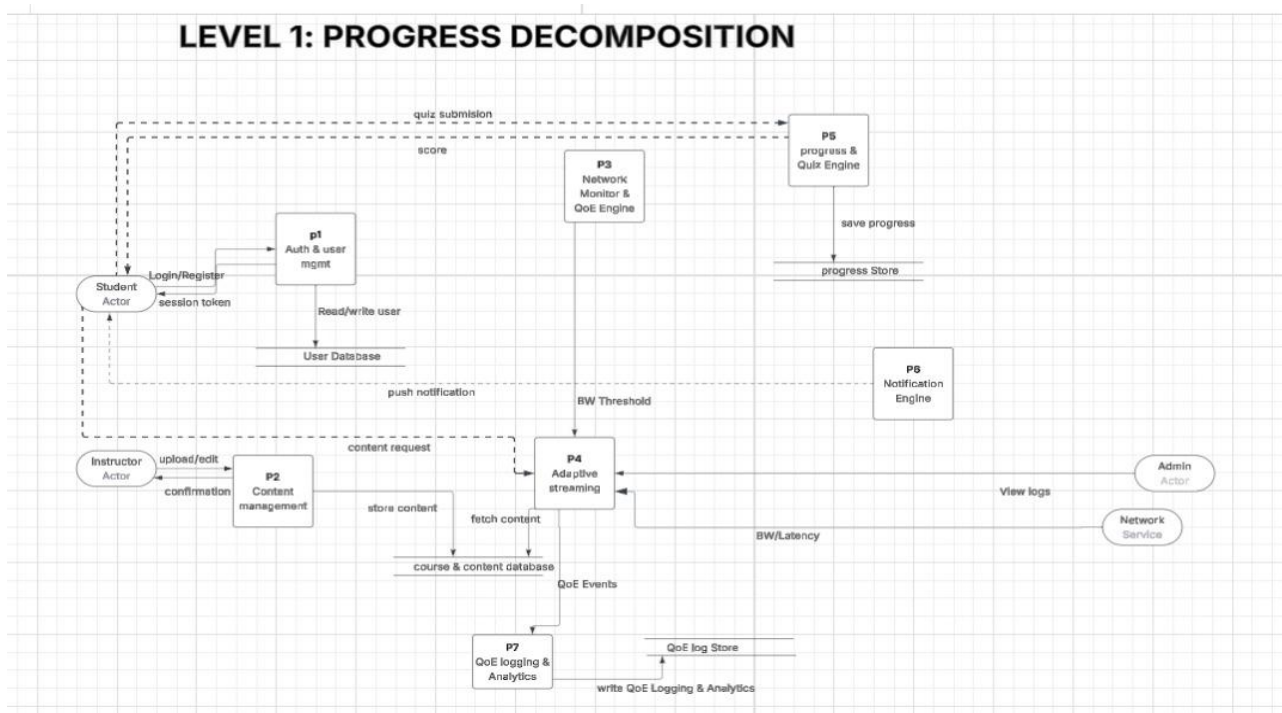
The Level 1 diagram breaks the system down into its internal parts. These parts are the seven processes and four data stores. The seven processes are P1, P2, P3, P4, P5, P6, and P7. The four data stores are

Processes

The seven processes and what they do are described below.

- **P1 – User Authentication:** This is the first process. It handles user registration and login. When a student or instructor enters their credentials, P1 checks them against the User Database (D1) and issues a session token to allow access. No other process works without this one.

- **P2 – Content Management:** This process handles everything related to course content. When an instructor uploads, edits, or deletes a course, P2 processes the request and saves the result to the Course and Content Store (D2). It also sends a trigger to P6 so that students are notified of new content.
- **P3 – Network Monitor and QoE Engine:** This process receives real-time network data from the Network Service external entity. It monitors the bandwidth, latency, and packet loss and calculates a Quality of Experience (QoE) score. It then sends a signal to P4 telling it what quality level to use for content delivery.
- **P4 – Adaptive Streaming:** This is the most important process in the system. It receives content requests from students and, based on the signal from P3, decides how to deliver the content. If the network is good, it streams high-quality video. If the network is poor, it switches to audio-only or text mode. It fetches content from D2 and logs quality events to P7.
- **P5 – Progress and Quiz Engine:** This process handles quiz submissions from students. When a student submits a quiz, P5 marks it and returns the score immediately. It also saves the student's completion percentage and time spent to the Progress Store
- **P6 – Notification Engine:** This process is triggered by P2 whenever new content is uploaded or a deadline is approaching. It sends push notifications to students to keep them informed without them having to check manually.
- **P7 – QoE Logging and Analytics:** This process records all quality-related events such as mode switches, buffering incidents, and session durations. It saves these records to the QoE Log Store and provides reports to the Admin when requested.



Data Stores

The four data stores and what they hold are described below.

- **User Database:** This store holds user credentials, roles, and session data. It is used by P1 to verify login information and assign the correct role to each user.

- **Course and Content Store:** This store holds all the course materials uploaded by instructors, including videos, notes, and quizzes. It is written to by P2 and read from by P4 when delivering content to students.
- **QoE Log Store:** This store holds records of all quality switch events, buffering incidents, and mode changes. It is written to by P7 and accessed by the Admin to review system performance.
- **Progress Store:** This store holds each student's quiz scores, completion percentages, and time spent per course. It is updated by P5 each time a student completes a quiz or activity.

Use Case Diagram

The use case diagram for the E-Learning Platform illustrates the interactions between three actors: **Student**, **Instructor**, and **Administrator** and the system. The system boundary is represented by a rectangle labeled "E-Learning Platform" which contains all use cases.

Actors

Student is the primary actor who interacts with the most use cases. The student is responsible for accessing course content, taking quizzes, and managing their learning experience.

Instructor is a primary actor who is responsible for managing course content and delivering live lectures to students.

Administrator is a secondary actor positioned on the right side of the diagram whose role is to manage the system, users and access control.

Use Cases and Relationships

Student Use Cases

The Student actor is connected to the following use cases:

Browse and access enrolled courses , allows the student to view and access their enrolled courses. It has two extend relationships: Download course materials which extends it when a student chooses to save content offline, and View progress per course which extends it to allow students to track their learning progress.

Attempt quizzes ,allows students to take quizzes. It includes Get automated score meaning every time a student submits a quiz, an automated score is immediately generated.

Watch recorded video lectures , allows students to view pre-recorded lectures. Receive in-app notifications extends this use case meaning students are notified when new video lectures are available.

Shared Use Cases (Student and Instructor)

Login : both Student and Instructor must log in to access the system. It includes Register with email or Password meaning a user must be registered before logging in. Reset Password extends Login for cases where the user forgets their password.

Receive in-app notifications : both Student and Instructor receive notifications about new content, quizzes and live sessions.

Instructor Use Cases

Create/Edit/Delete course content : allows the Instructor to manage all course materials including videos, notes and quizzes.

Initiate live video lecture extends this use case when the instructor decides to conduct a live session.

Administrator Use Cases

The Administrator is connected to three use cases with plain association lines:

Manage Users : allows the Admin to add, edit or remove users from the system

Assign Roles:allows the Admin to assign roles such as Student, Instructor or Admin to registered users

Monitor System :allows the Admin to oversee system performance and uptime

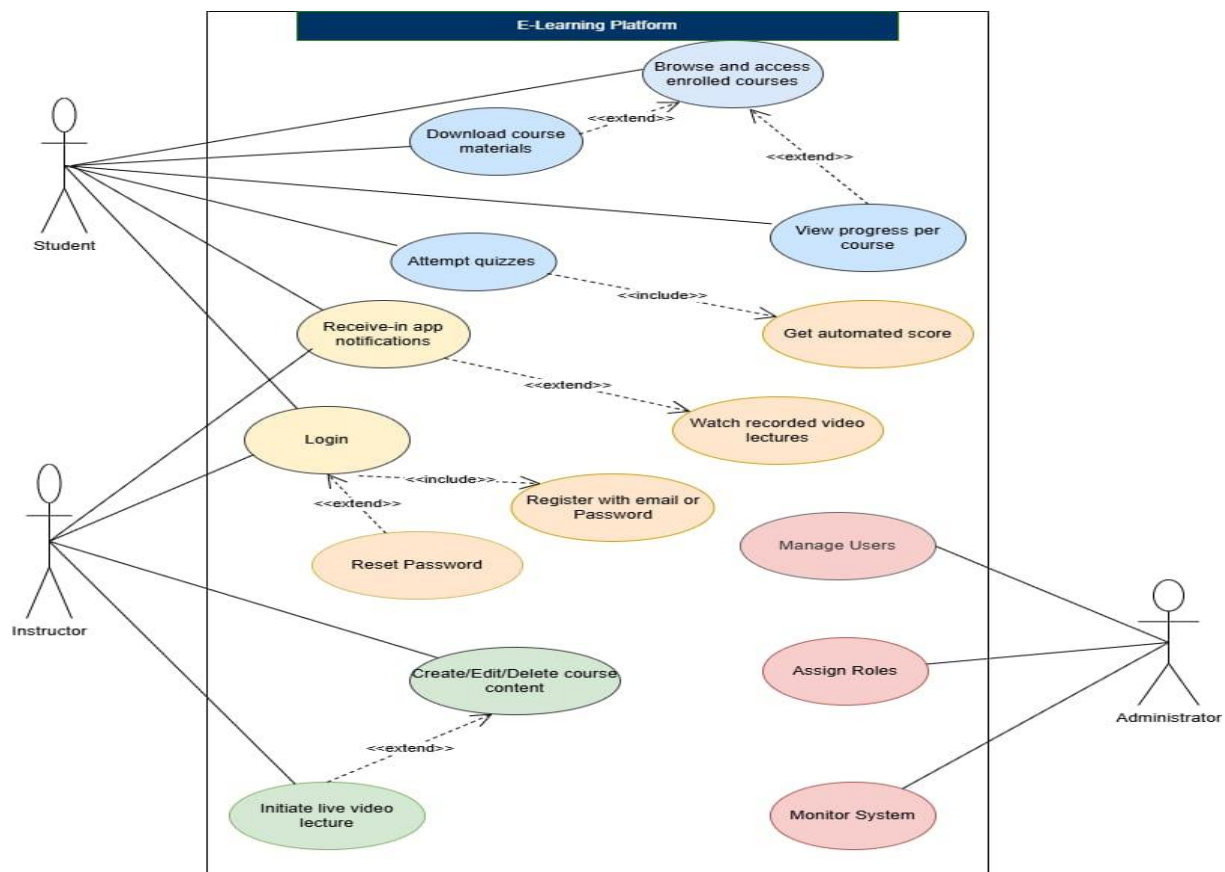


Figure 3 Use Case Diagram

Sequence Diagram

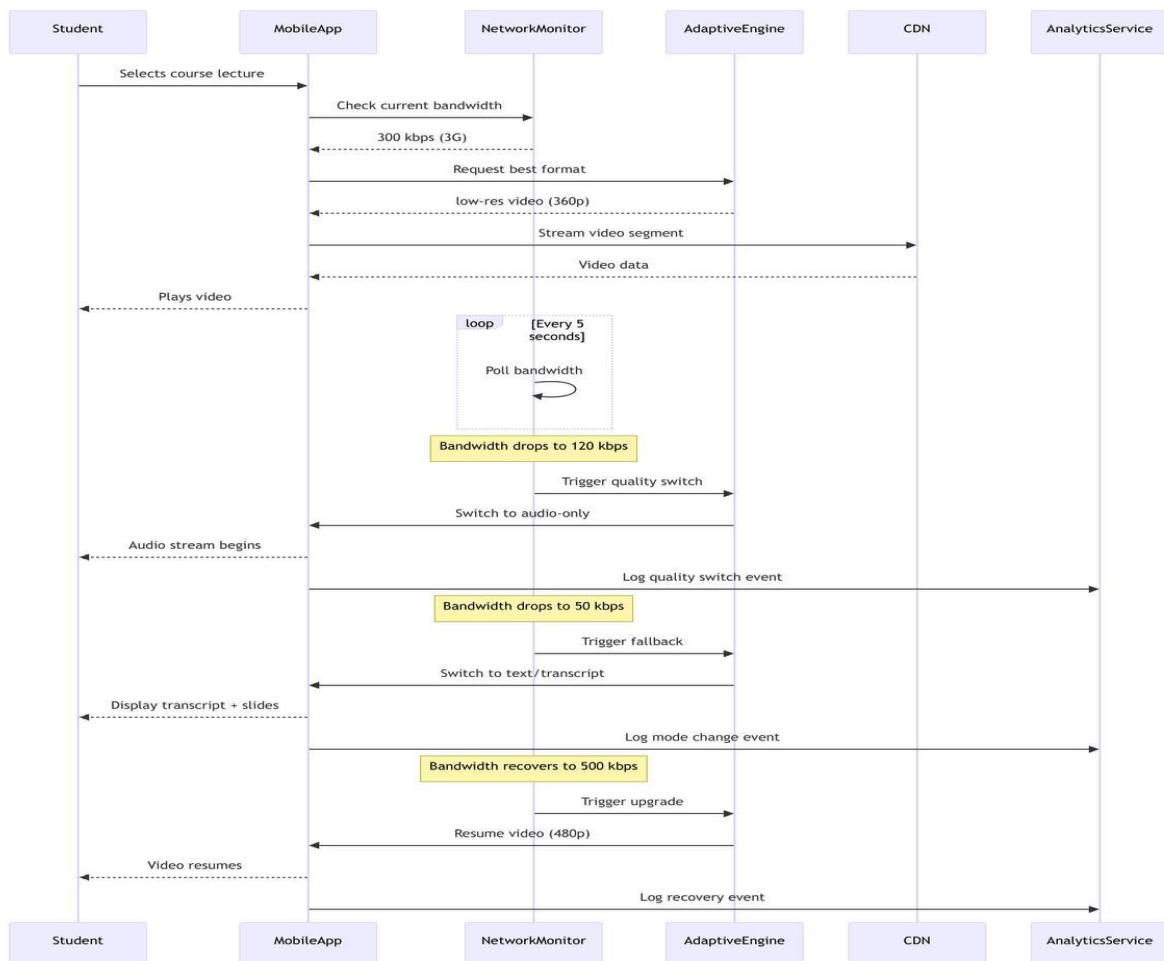
A sequence diagram is a UML (Unified Modeling Language) tool that shows how different parts of a system communicate with each other, and in what order. It focuses on the dynamic behavior of a system—meaning it shows the flow of messages over time, not just what components exist.

while a class diagram is like a “map of the system structure,” a sequence diagram is like a “story of what happens step-by-step.” It helps teams understand the real interactions between users, apps, servers, and services.

Scenario S1: Adaptive Content Delivery

Actors: Student, Mobile App, Network Monitor, Adaptive Engine, CDN, Analytics Service

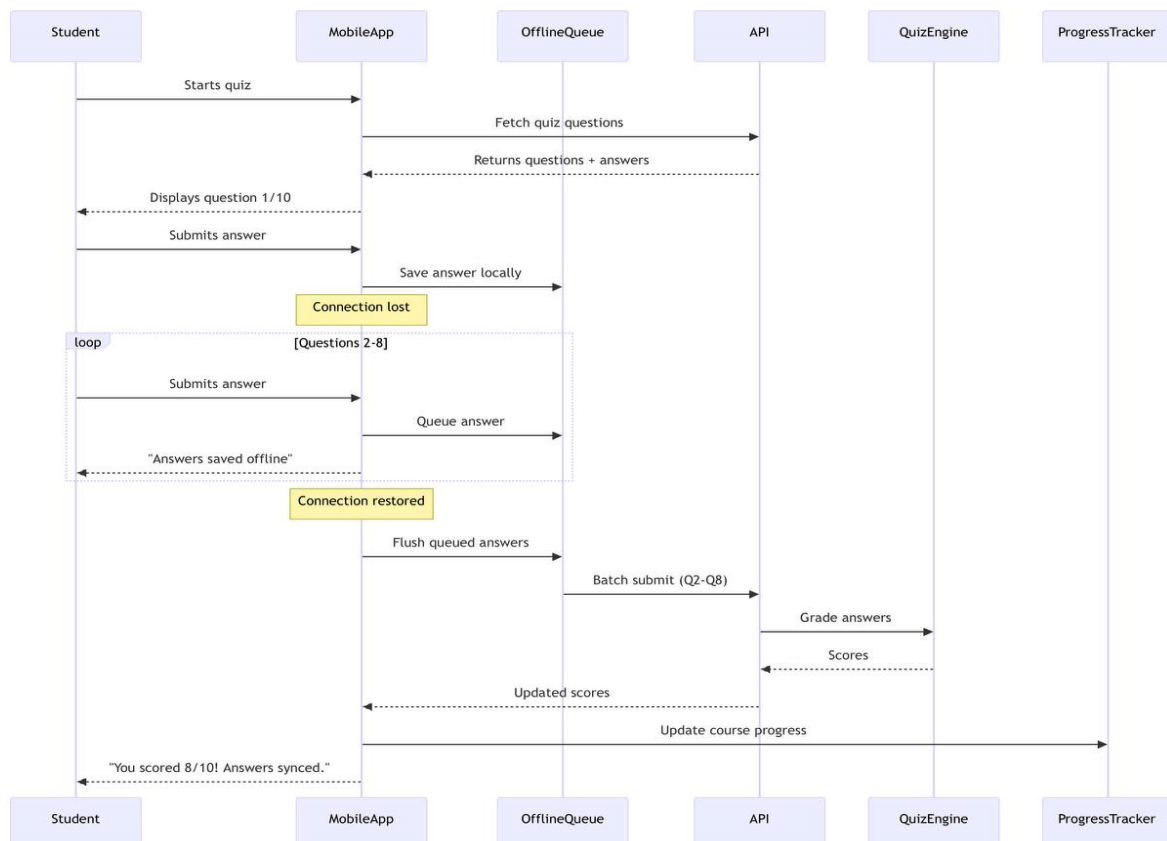
Simple Flow: A student opens a lecture on the app. The app first checks their internet speed and starts with a suitable video quality, like 360p for good bandwidth. As they watch, the app keeps checking the connection. If the signal weakens, it seamlessly switches the experience—first to audio-only, and if it gets worse, to a text transcript. When the signal improves again, it upgrades back to higher-quality video. Every switch is quietly recorded for analysis. The student never faces buffering or errors; the content just adapts.



Scenario S2: Quiz Taking When Internet Drops

Actors: Student, Mobile App, Offline Queue, API, Quiz Engine, Progress Tracker

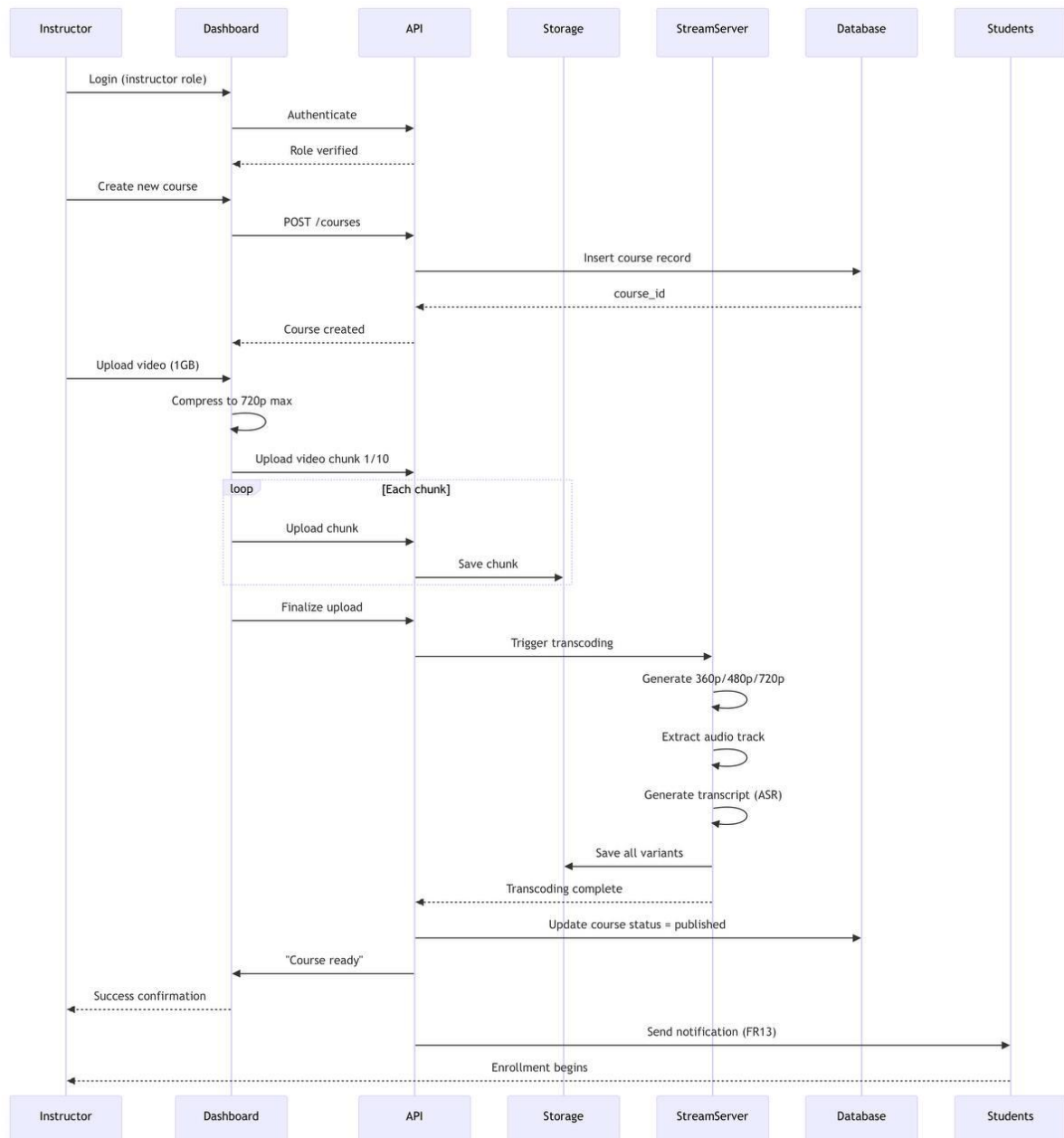
Simple Flow: A student starts a quiz with an active internet connection. If the connection suddenly drops mid-quiz, the app doesn't stop. It switches to offline mode, letting the student continue answering. Each response is saved locally in an offline queue, and the student is notified their work is being saved offline. Later, when they reconnect to Wi-Fi, the app automatically sends all saved answers to the server in one batch. The server grades them, updates progress, and shows the final score—ensuring no work is lost.



Scenario S3: Instructor Course Creation with Video Upload

Actors: Instructor, Dashboard, API, Storage, Streaming Server, Database, Students

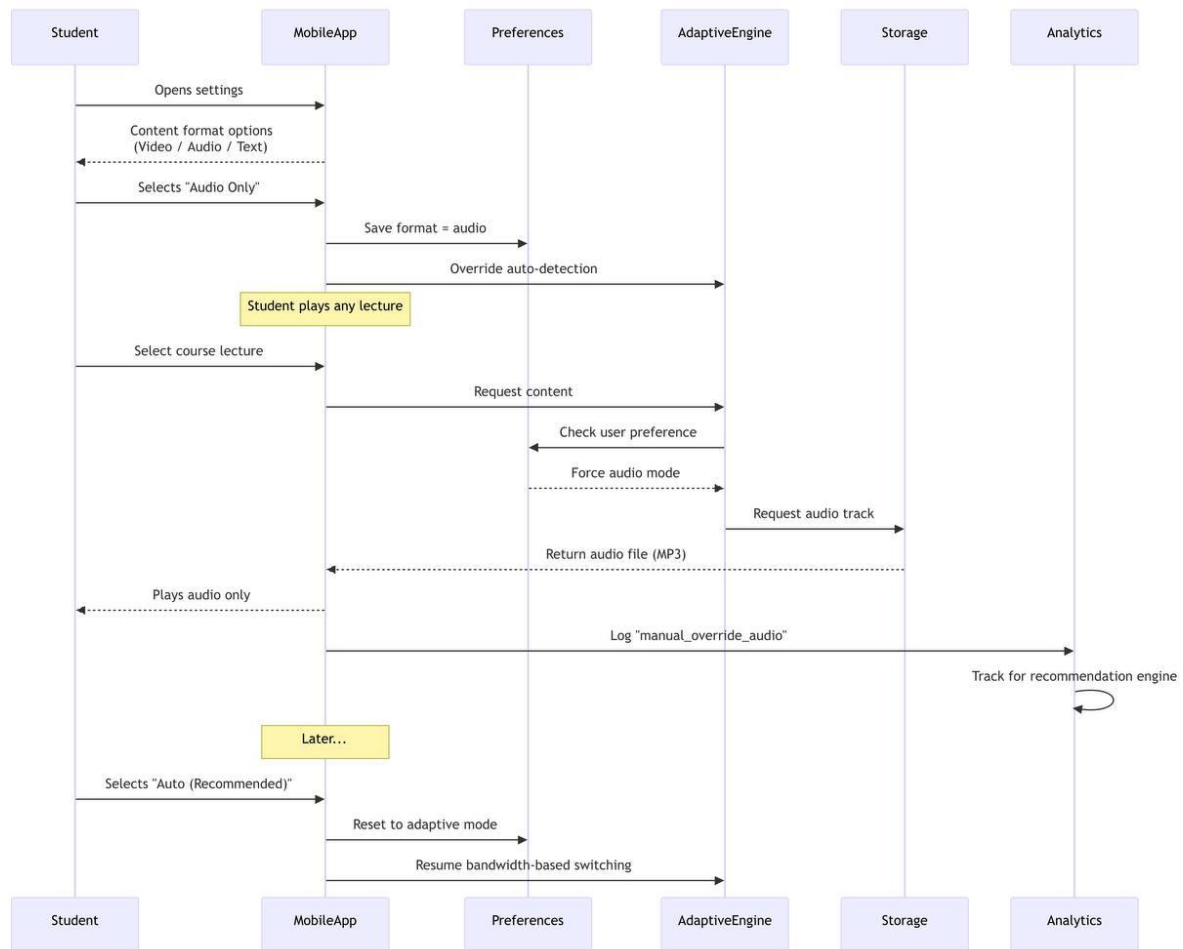
Simple Flow: An instructor logs in, creates a new course by filling in details, and uploads a large video file. To ensure reliability, the app breaks the video into smaller pieces and uploads them one by one. After upload, the system processes the video: it generates multiple quality versions (like 360p, 480p), creates an audio track, and uses speech recognition to make a text transcript. Once everything is saved, the course is published, and enrolled students get a notification. The instructor sees a "Course ready" confirmation.



Scenario S4: Manual Format Selection by Student

Actors: Student, Mobile App, Preferences, Adaptive Engine, Storage, Analytics

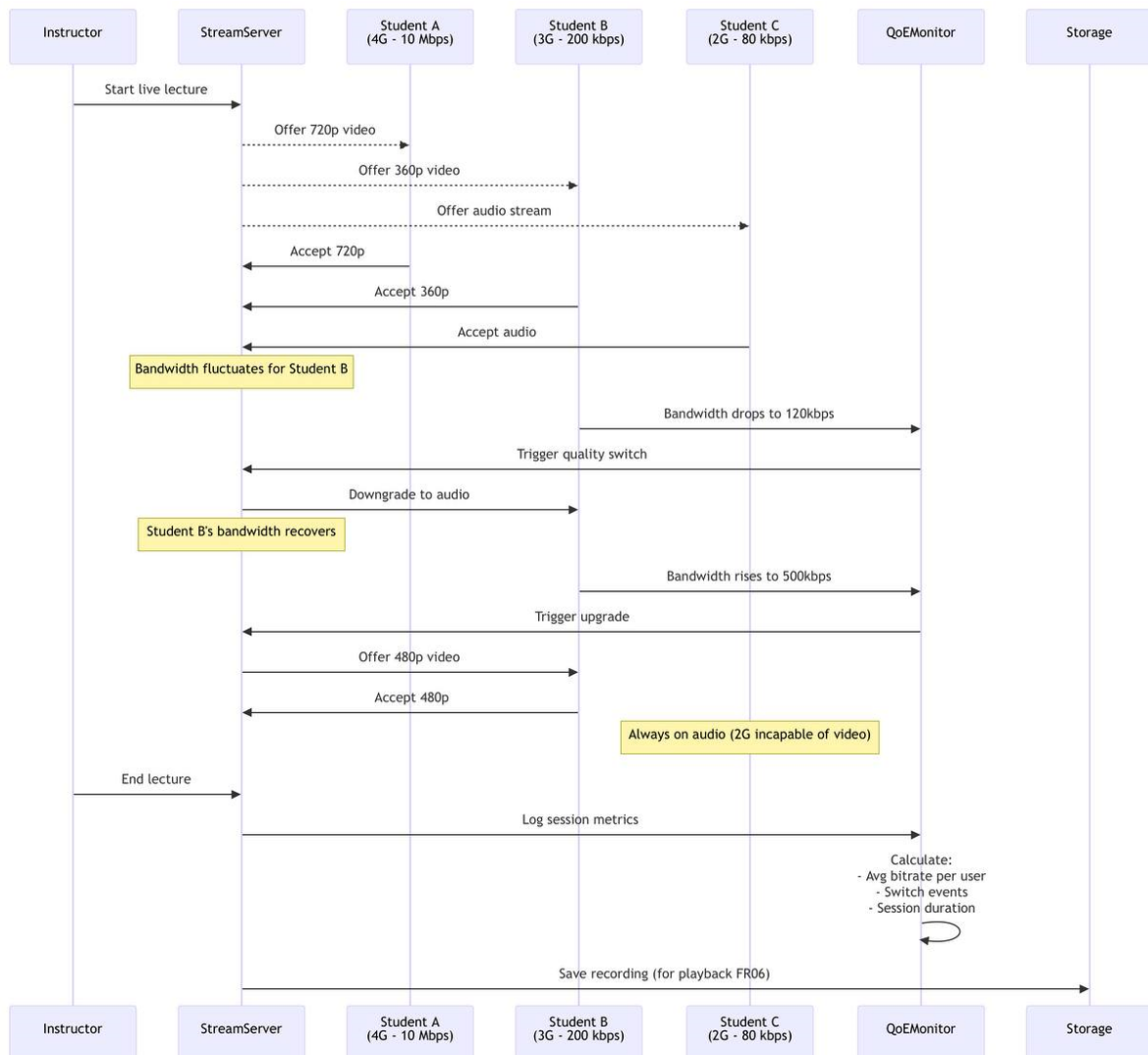
Simple Flow: A student can choose how they want to consume content, especially to save data. In the app settings, they select an option like “Audio Only.” This preference is saved. When they open a lecture, the app skips its usual internet check and directly fetches the audio version from storage. The lecture plays as audio, ideal for commuting. The choice is logged for analytics. If they later switch back to “Auto,” the app removes the manual setting and returns to automatic quality adjustments based on network conditions.



Scenario S5: Concurrent Live Lecture

Actors: Instructor, Streaming Server, Students (A, B, C on 4G/3G/2G), Quality Monitor

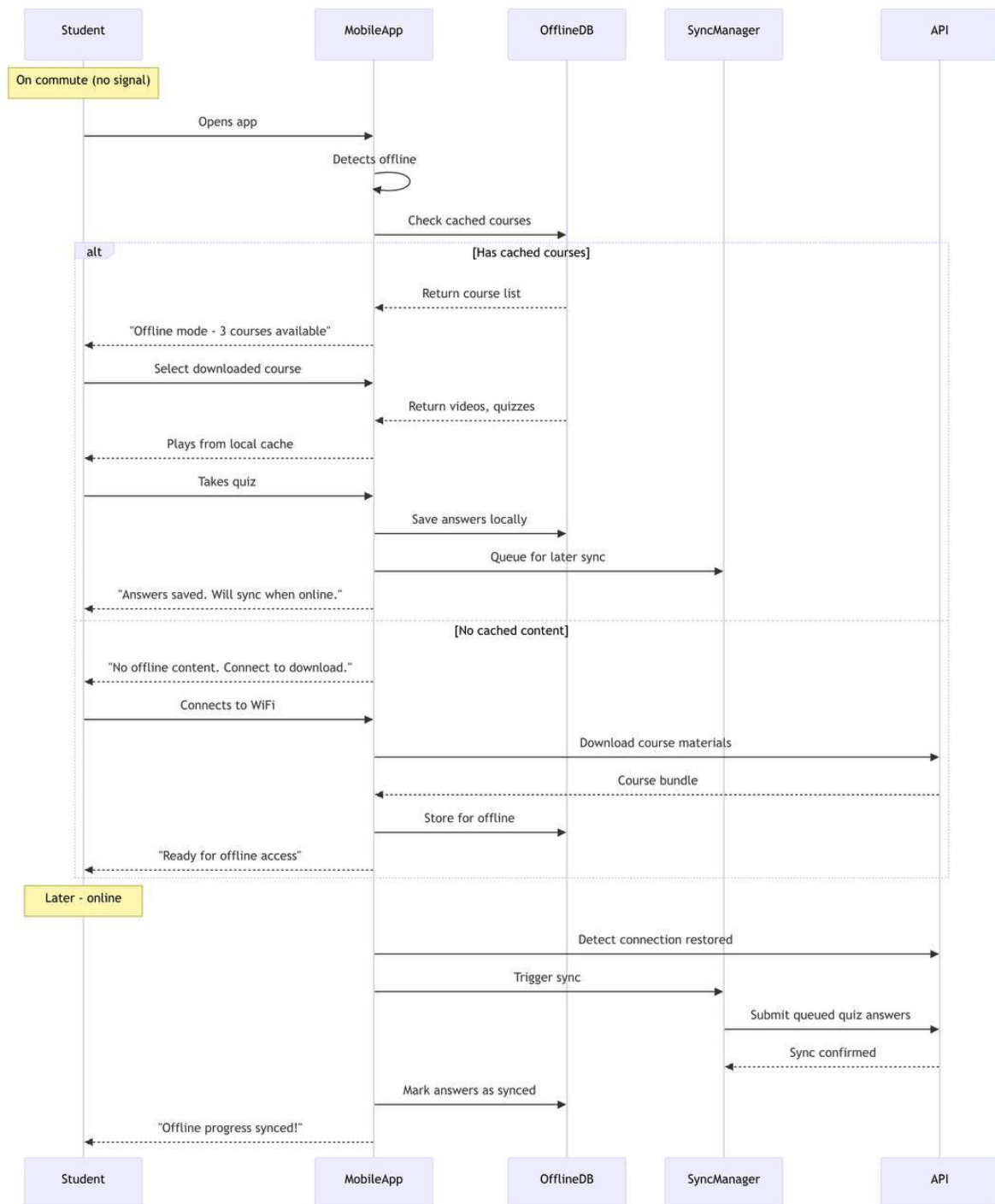
Simple Flow: An instructor starts a live lecture. Three students join with different connections: strong 4G, moderate 3G, and weak 2G. The server gives each the best matching stream—high-quality video, lower-quality video, and audio-only. During the session, if Student B's connection worsens, a monitor detects it and tells the server to downgrade their stream to audio-only within seconds. When it improves, they are upgraded back. Student C stays on audio. After the lecture ends, a recording is saved for later viewing in any format.



Scenario S6: Complete Offline Mode

Actors: Student, Mobile App, Offline Database, Sync Manager, API

Simple Flow: A student downloads courses on Wi-Fi before a commute. Later, with no signal, they open the app. It detects the offline state and shows the downloaded content with an “Offline mode” banner. They can watch videos and take quizzes, with all answers saved locally. When they eventually reach Wi-Fi again, the app automatically reconnects. A sync manager sends the saved quiz answers to the server, which confirms receipt. The student gets a “Offline progress synced!” message, guaranteeing nothing was lost.

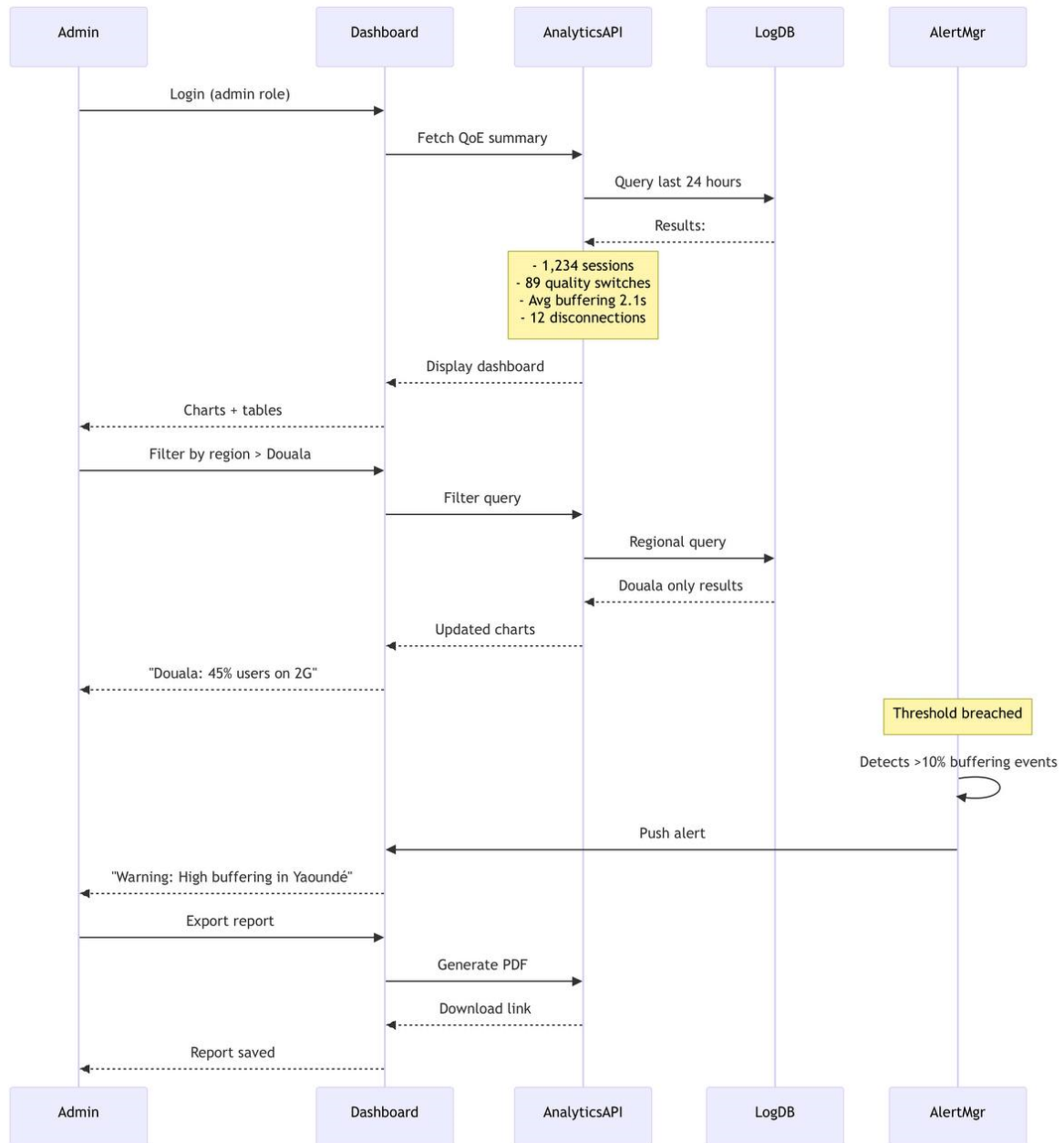


Scenario S7: Quality of Experience Analytics

Actors: Administrator, Dashboard, Analytics API, Log Database, Alert Manager

Simple Flow: An administrator logs into a dashboard to view user experience metrics. The system shows data from the last 24 hours: session counts, quality switches, average buffering,

and disconnections. The admin can filter by region, like seeing how many users are on 2G in Douala. If something is abnormal—like high buffering in Yaoundé—the system triggers an immediate alert. Finally, the admin can export a PDF report of these metrics to share with the team, helping identify where infrastructure improvements are needed.



Class Diagram

The class diagram for the E-Learning Platform defines the static structure of the system showing the classes, their attributes, methods and the relationships between them. The diagram contains nine classes: User, Student, Instructor, Course, Content, Quiz, QuizResult, Notification, RecommendationEngine and StreamingSession.

Classes

User

The User class is the base abstract class for all users in the system. It contains common attributes shared by all users including `userId`, `email`, `passwordHash` and `role` which can be Student, Instructor or Admin. Its methods include `login()`, `resetPassword()`, `selectContentFormat()` and `receiveNotification()`.

Student

The Student class extends the User class through an inheritance relationship. It contains attributes specific to students such as `enrolledCourses` and `progressMap`. Its methods include `accessCourse()`, `browseCourses()`, `attemptQuiz()`, `downloadContent()`, `viewRecommendations()`, `viewDataUsage()` and `trackProgress()`.

Instructor

The Instructor class also extends the User class through inheritance. It contains the attribute `createdCourses` and methods including `createCourse()`, `editCourse()`, `deleteCourse()`, `initiateLiveLecture()` and `uploadContent()`.

Course

The Course class represents a course in the system. Its attributes include `courseId`, `title`, `instructor`, `contents` and `quizzes`. Its methods are `getContent()`, `addContent()` and `removeContent()`. The Course class has an aggregation relationship with Instructor (1 to 1,n) meaning one instructor can create one or more courses. It also has a composition relationship with Content (1 to 0,n) meaning a course contains zero or more content items that cannot exist without the course.

Content

The Content class represents learning materials within a course. Its attributes include `contentId`, `type` which can be Video, Audio, Note or Transcript, `isDownloadable` and `isLive`. Its methods are `stream()` and `download()`. It has an association relationship with StreamingSession.

Quiz

The Quiz class represents assessments within a course. Its attributes include `quizId`, `courseId`, `title`, `questions`, `deadline`, `totalMarks` and `passMark`. Its methods are `submit()` and `autoScore()`. It has a composition relationship with QuizResult (1 to 0,n).

QuizResult

The QuizResult class stores the outcome of a student's quiz attempt. Its attributes include resultId, studentId, quizId, score, isPassed and submittedAt. Its methods are getScore() and hasPassed(). It has an association with the Student class (1 to 0,n).

Notification

The Notification class manages in-app alerts sent to users. Its attributes include type which can be NewContent, QuizDeadline or LiveSession, message and timestamp. Its methods are send() and markRead(). It has an association with the User class (1 to 0,n) meaning one user can receive zero or more notifications.

RecommendationEngine

The RecommendationEngine class handles course recommendations for students. Its attributes include studentHistory stored as a Map. Its methods are recommend() and updateHistory(). It has associations with both Student and Course classes.

StreamingSession

The StreamingSession class manages the technical aspects of content delivery. Its attributes include bandwidth, latency, packetLoss, currentMode which can be Video, Audio or Text, and dataUsage. Its methods include adjustQuality(), switchToAudioOnly(), switchToTextMode() and logQoEMetrics(). It has an association with the Content class (1 to 1).

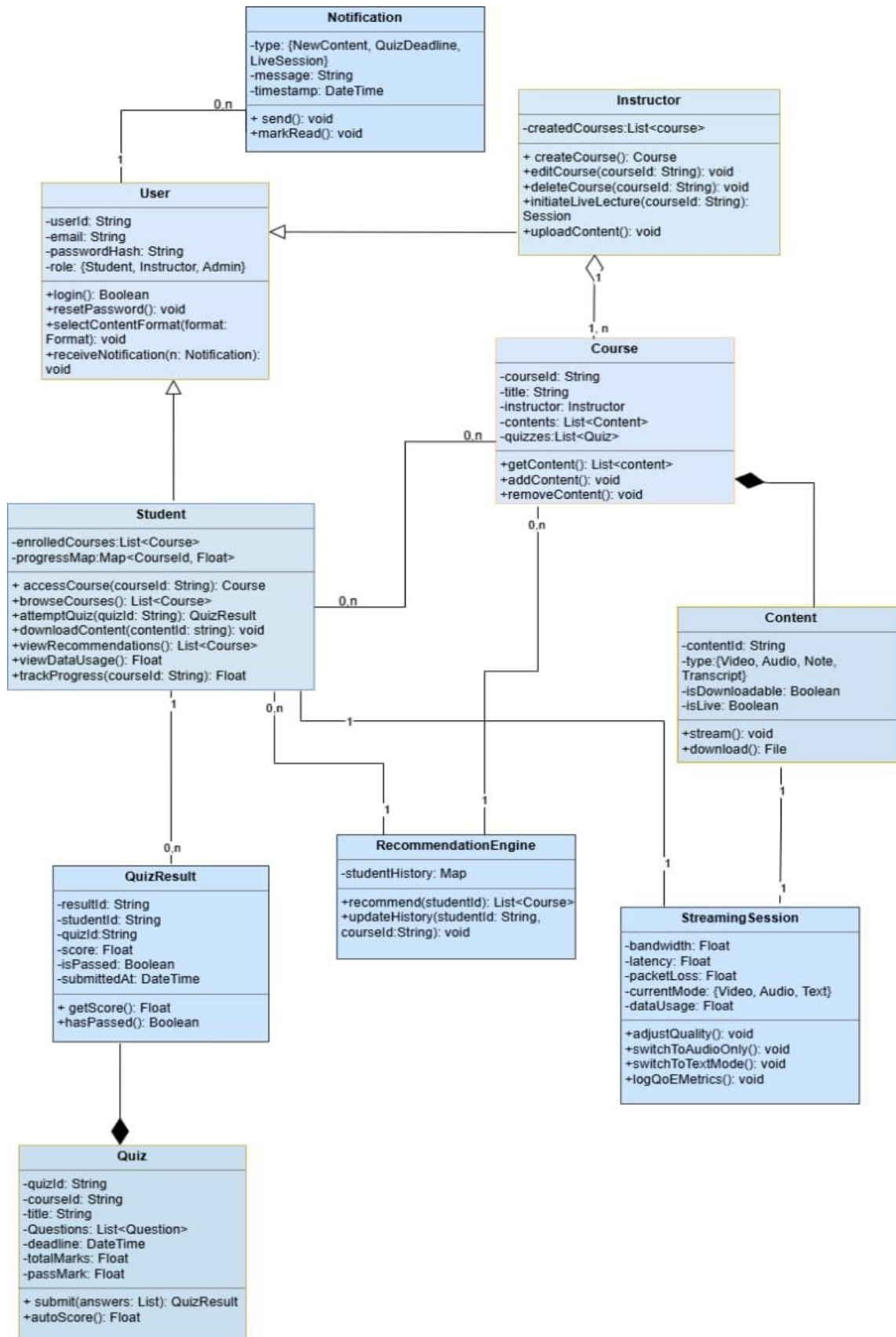


Figure 4 Class Diagram

Deployment Diagram

A Deployment Diagram is a type of Structural UML Diagram that shows the physical deployment of software components on hardware nodes. It illustrates the mapping of software components onto the physical resources of a system, such as servers, processors, storage devices, and network infrastructure.

A Deployment Diagram shows how the software design turns into the actual physical system where the software will run. They show where software components are placed on hardware devices and shows how they connect with each other. This diagram helps visualize how the software will operate across different devices.

The Different Tiers of our Deployment Diagram are demonstrated below.

Client tier

React builds to static files. Static files are cheap to serve.

Vercel or Netlify free tier handles the React frontend at zero cost. We can push our code, they are built and served globally.

On-premise server

- **Replace PostgreSQL with SQLite for early development.** PostgreSQL is excellent but it's a separate process with its own memory overhead. SQLite runs inside our Node process, writes to a single file, needs zero configuration, and handles hundreds of concurrent users comfortably for a project at this stage. Migrating to PostgreSQL when the system scales can be handled with other entities like Prisma or Drizzle.

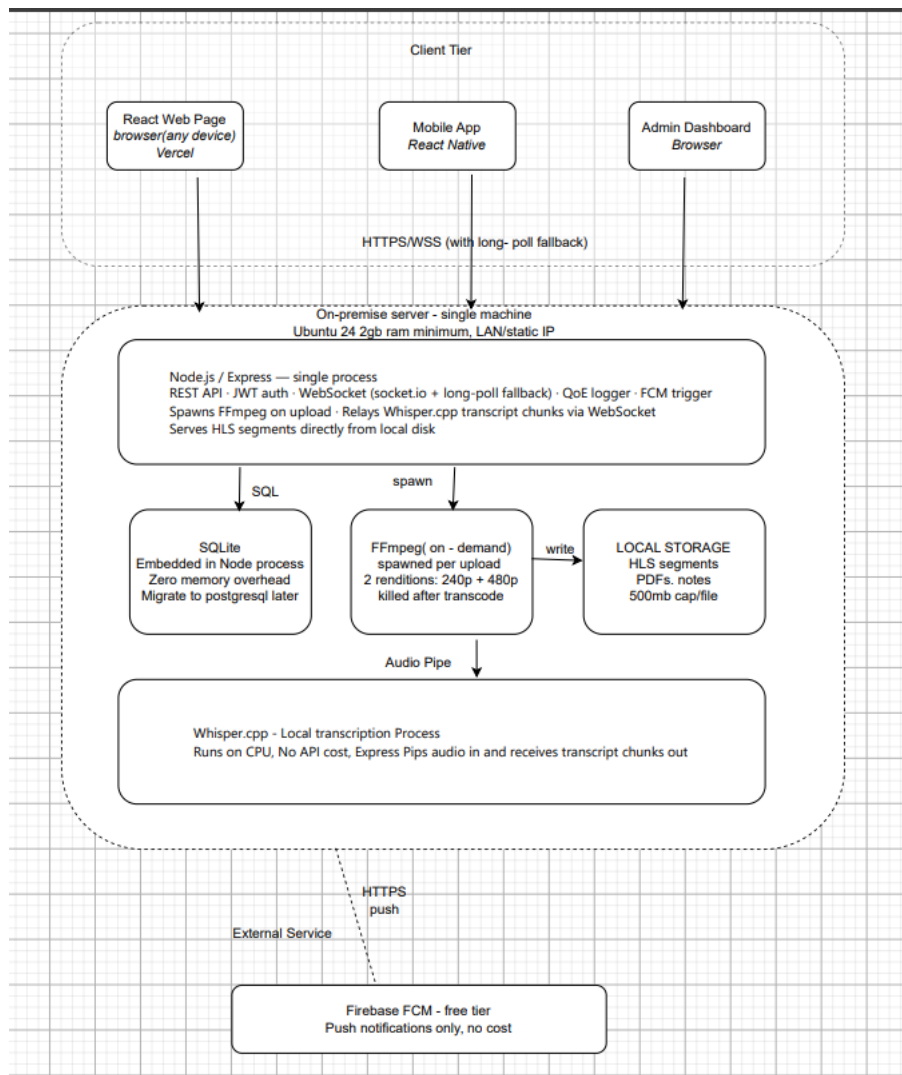
- **FFmpeg runs as a background job, not a parallel service.** Instead of FFmpeg being always-on, Express spawns it as a child process only when a video is uploaded, then kills it when transcoding is done. This means FFmpeg uses CPU and memory only when needed rather than continuously. This also reduces the number of renditions — instead of 360p, 480p, 720p, we produce just two: a low-quality (240p, ~150kbps) and a standard-quality (480p, ~800kbps). This cuts transcoding time and storage in half.

- **Local file storage stays, but we add a size discipline.** No cloud storage budget means videos live on the same disk as web server. We need to enforce upload limits — cap video uploads at 500MB per file, and set a total storage quota per course. This isn't just a resource decision, it's a requirement we add to the system (one of our open issues from the SRS actually touches this).

External services

Firebase FCM . It has a permanent free tier with no meaningful limits for a project at this scale.

The Transcription API gets deferred or swapped. Google Speech-to-Text is metered — we pay per minute of audio. For a low-budget setup, this becomes expensive fast during live sessions. Two better options: first, use Whisper.cpp running locally on web server — it's Open AI's Whisper model compiled to run on CPU, free, no API calls, and good enough for lecture transcription. The trade-off is CPU load during live sessions. Second, defer transcription entirely to Phase 2 and replace it with a simpler live text chat where the instructor types key points — zero infrastructure cost, and web survey showed 34.3% valued transcription, not 100%.



CONCLUSION:

This task provided a comprehensive foundation for understanding and designing the architecture of the mobile application for an Adaptive E-learning Software. Through the development of various system modelling diagrams—including the Context Diagram, Data Flow Diagram (DFD), Use Case Diagram, Sequence Diagram, Class Diagram, and The Deployment Diagram, we were able to lay down the foundation of the system which facilitates next steps of the development process.